On the Limitations of Aggregate Tracking-Error Metrics for Evaluating Weather Robustness in Autonomous Row-Following Robots

Sarvesh Satish
Heartfulness International School
Hyderabad, India
sarveshsatish09@gmail.com

Sumit Soman
Senior Member, IEEE
sumitsoman@ieee.org

Kiran Siripuri
Symbiosis Institute of Technology
Hyderabad, India
skiran@sithyd.siu.edu.in

Abstract

Vision-guided agricultural robots must continue following crop rows as visual conditions degrade, yet robustness is often summarized by a single tracking-error statistic. This paper demonstrates, in simulation, that such a statistic can rank conditions incorrectly. A differential-drive robot with a fixed-threshold HSV row detector was simulated in Webots for 20 s of simulated time under clear weather (NORMAL), synthetic fog, and synthetic rain. Each steering command was also sent to a physical Arduino Uno, and an identifier-matched acknowledgement was used to measure the communication path. Fog reduced the mean number of detections per step from 4.5 to 0.6 and increased the RMS pixel error from 24.1 px to 46.9 px. Rain produced the lowest and most stable pixel error of the three conditions (16.1 px mean), yet the detector reported exactly five detections in all 624 steps, and a majority of detections had the same fixed width produced by the detector's oversized-region split. These diagnostics indicate that the global brightness reduction in the rain model moved the detector's effective dark-pixel threshold so that the mask merged into one static region. Thus, the low error reflects detector lock-on rather than accurate row following. We recommend reporting detection-stability diagnostics alongside tracking error when evaluating fixed-threshold perception pipelines under weather perturbations.

Keywords agricultural robotics · row-following · machine vision · Webots · adverse weather · evaluation metrics · failure diagnosis

1 Introduction

Agricultural robots operating in orchards, vineyards, and row crops often rely on a forward-facing camera to locate crop rows and steer along them, particularly on small platforms for which LiDAR and RTK-GNSS may be costly. Classical vision-based row following has been studied for two decades [1], and recent surveys cover both threshold-based and learned detectors [2]. Visual-servoing approaches based on monocular cues have also been demonstrated for row-crop navigation [3].

The effect of adverse weather on camera-based perception has been studied extensively in autonomous driving, where rain and fog can degrade detection performance [4]. Synthetic image corruptions, including fog, are commonly used to benchmark perception robustness [5, 6]. In row-following, robustness can be summarized using aggregate tracking-error statistics such as mean or RMS lateral error. Such metrics implicitly assume that a small error indicates that the perception system is tracking the scene. This assumption can fail when the perception output becomes insensitive to the scene: a detector that repeatedly outputs a nearly constant position can yield a small and stable error without tracking the underlying row geometry.

This paper examines this failure mode in a controlled simulation. We evaluate a fixed-threshold HSV row detector under clear, foggy, and rainy conditions and compare tracking error with three detection-stability diagnostics. The contributions are:

1. A documented case in which a synthetic rain perturbation yields the lowest tracking error of three conditions while the detector is not following the scene, so that the aggregate metric inverts the true ranking.
2. Three inexpensive diagnostics that expose the failure: per-step detection count and its variance, the distribution of detection widths, and track lifespan.
3. A quantitative account of how a global brightness reduction shifts the effective HSV threshold of the detector, which explains the failure.
4. A timing instrumentation of the full loop, with an identifier-matched serial acknowledgement, which separates communication latency from perception latency.

The results concern one fixed-threshold detector, synthetic image-space weather perturbations, and one 20 s log per condition. Section 7 states the scope and limitations of the evidence.

2 Related Work

Webots [7] provides simulation of wheeled robots, camera sensors, and external controllers and is used to prototype robotic systems before field trials. Åstrand and Baerveldt [1] developed an early complete row-following system based on Hough-transform row detection and steering from lateral offset, which is conceptually related to the pixel-error control used here. Bai et al. [2] survey vision-based navigation for agricultural vehicles, including colour and threshold segmentation, Hough-based methods, and learned detectors. Ahmadi et al. [3] demonstrate monocular visual-servoing-based navigation for row-crop monitoring.

Weather robustness has been studied most extensively for road vehicles. Zhang et al. [4] survey the effects of rain, fog, and snow on camera, LiDAR, and radar perception and discuss mitigation approaches such as learned de-hazing and de-raining. Image-space corruption benchmarks [5] and their use for object detection under simulated winter conditions [6] provide relevant precedents for synthetic weather perturbations. Fog has also been synthesized using physically motivated attenuation models [8], while rain has been modeled as a combination of photometric and dynamic effects [9]. In contrast, the filters used here are intentionally simple image-space transformations, as discussed in Section 7. The present work therefore focuses on a narrower question: whether an aggregate tracking-error statistic can become misleading when used to evaluate a classical perception pipeline under such perturbations.

3 System Architecture and Methodology

3.1 System Overview

Fig. 1 shows the data flow. A Webots controller written in Python reads the camera frame, applies the weather filter, runs the vision pipeline, and computes a steering command u and a forward speed v. The same command is applied directly to the simulated wheel motors and is also transmitted over a USB serial link to a physical Arduino Uno. The Arduino actuates a steering servo and returns an acknowledgement. The acknowledgement is logged for latency measurement but is not fed back to the simulated robot. The simulated closed loop is therefore camera, vision, controller, wheels, and camera; the Arduino is a hardware-coupled branch of this loop whose communication delay is measured but does not alter the simulated vehicle.

3.2 Simulation Platform

The platform is a differential-drive Pioneer-class robot in Webots R2025a with a forward-facing RGB camera (Fig. 2) in a field containing 15 trees arranged in rows. The controller uses a fixed simulation step of 32 ms. The camera resolution is 640×480 pixels with a horizontal field of view of 1.9 rad. Serial communication uses 115200 baud. Table 1 lists the fixed parameters, which were held constant across conditions.

3.3 Vision-Based Row Detection

Each frame is converted to HSV, and a binary mask is formed as the union of three conditions: a green-foliage range (H 25–100, S 25–255, V 20–220), a dark range (any hue, S $\geq$ 15, V $\leq$ 125), and a dark-trunk rule (V $<$ 115 and S $>$ 15, or V $<$ 80 and S $<$ 100). The mask is restricted to the region of interest and cleaned with a morphological opening and a vertical closing. Mask pixels are counted per column, the profile is smoothed, and contiguous columns above a threshold are grouped into candidate regions. A region wider than the maximum tree width is not rejected: it is divided into consecutive pieces of 135 px (the split piece width), and each piece is reduced to the bounding box of its mask

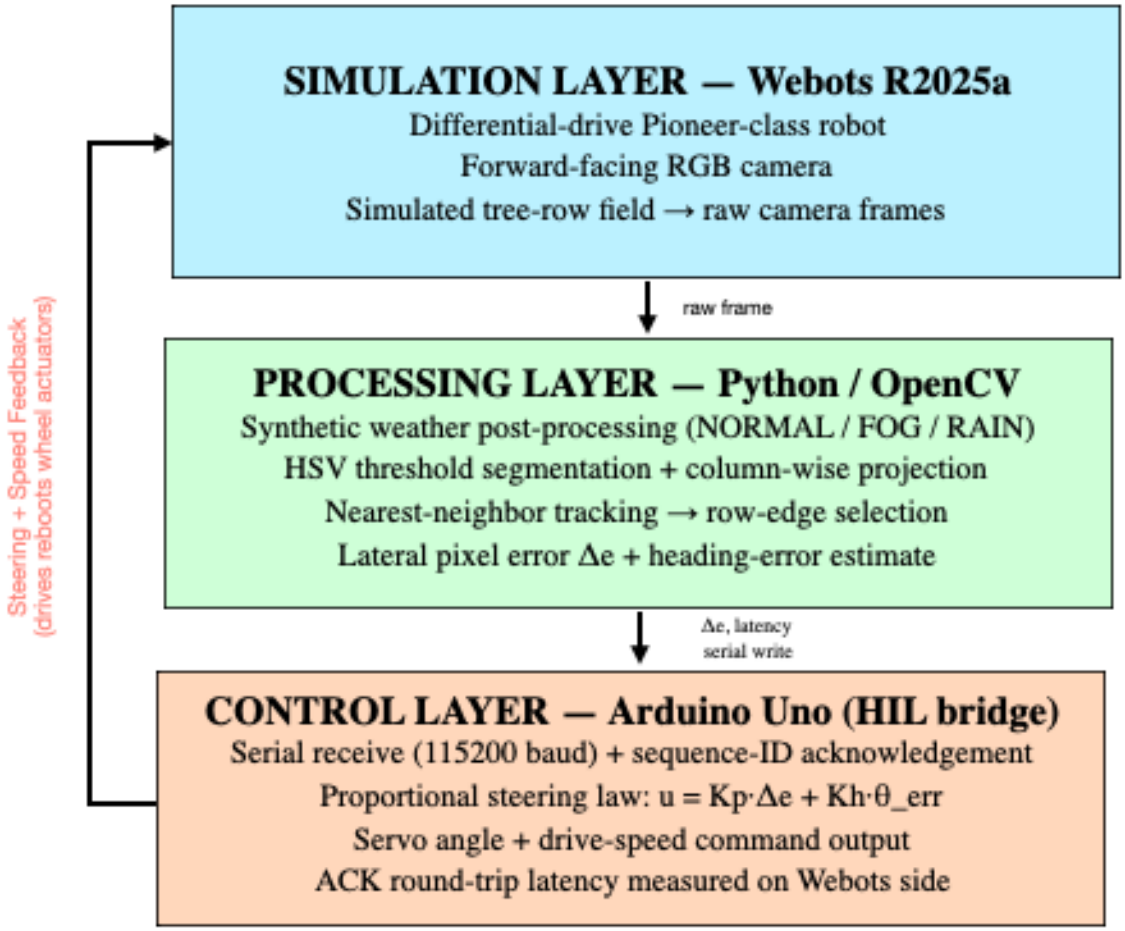


Figure 1: Data flow. The Python controller drives the simulated wheels directly and sends the same command to the Arduino. The acknowledgement is logged only.

pixels. Pieces are tested against the minimum size limits but not against the aspect-ratio test applied to undivided regions. Detections closer than 25 px to a larger detection are removed. Detections are associated between frames by nearest-neighbour matching within 90 px, and a track is dropped after 12 unmatched frames.

Detections whose centres lie left or right of the image centre are candidates for the left and right row, respectively. The *row reference* of each side is the candidate whose centre is nearest to the expected position (25% or 75% of image width), provided it lies within the acceptance window. When both references exist, the path centre X_{path} is their midpoint. When only one exists, X_{path} is offset from it by half the most recently observed row spacing, or by half the nominal spacing if none has been observed (the single-row estimate). When neither exists, the last path centre is reused, and if none has ever been computed the robot stops.

3.4 Control Law

Let W be the image width. The lateral pixel error and the heading-error estimate are

$$e = X_{\text{path}} - \tfrac{W}{2}, \tag{1}$$

$$\theta_k = \tfrac{180}{\pi}\ \operatorname{atan2}\big(X_{\text{path},k} - X_{\text{path},k-1},\ W\big), \tag{2}$$

so θ is the frame-to-frame change of the path centre expressed as an angle, not a measured vehicle heading. The steering command is

$$u = \operatorname{clip}\Big(10\,K_p\,\frac{e}{W/2} + K_h\,\theta,\ -u_{\max},\ u_{\max}\Big), \tag{3}$$

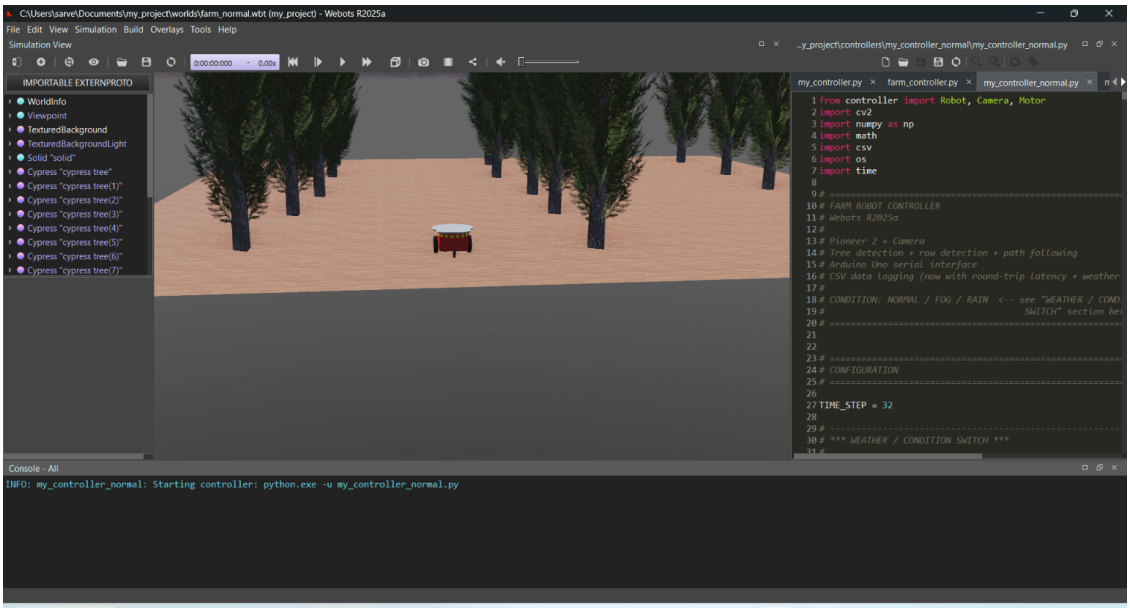


Figure 2: Webots environment: the robot in the simulated tree-row field, with the controller code.

Table 1: Fixed parameters (identical in all conditions)

Parameter	**Value**
Simulation step	32 ms
Run length	20 s (624 steps)
Serial rate / ACK timeout	115200 baud / 50 ms
Pixel gain K_p, heading gain K_h	0.035, 0.20
Steering limit $u_{\max}$	1.5
Base speed v_0 / speed floor	3.0 / 1.4 rad/s
Wheel speed limit	6.0 rad/s
Differential factor c	1.2
Servo angle	$90^\circ + 18^\circ u$ (45–135°)
Region of interest (rows)	2%–72% of image height
Min. tree width/height/area	7/20 px, 80 px^2
Max. tree width/height	180/430 px
Split piece width	135 px (0.75 of max. width)
Duplicate-suppression distance	25 px
Expected row positions	25% and 75% of image width
Row acceptance window	$\pm 28\%$ of image width
Tracker association / max. age	90 px / 12 frames

the forward speed is

$$v = \mathrm{clip}\Big(v_0\big(1 - 0.55\min(1, \tfrac{|e|}{0.45W})\big),\ 1.4,\ v_0\Big), \tag{4}$$

and the wheel angular velocities are $\omega_L = \mathrm{clip}(v - c\,u)$ and $\omega_R = \mathrm{clip}(v + c\,u)$ with limit ± 6 rad/s. Because v depends on e, equal simulated time does not imply equal distance travelled.

3.5 Latency Instrumentation

Each control step logs four durations: image capture, vision processing (mask through steering computation, excluding the weather filter), serial round-trip, and loop interval. The round-trip time is measured from the serial write to receipt of the corresponding acknowledgement line, with a 50 ms timeout. Each command carries a monotonically increasing sequence identifier, and an acknowledgement counts as successful only if it echoes the current identifier. The loop interval is the wall-clock time between the starts of consecutive iterations and therefore includes the simulator step, the weather filter, the serial exchange, display, and file logging.

Table 2: Summary of results

Metric	NORMAL	FOG	RAIN
Steps	624	624	624
Mean $\|e\|$ (px)	20.04	28.04	16.05*
RMS e (px)	24.12	46.94	16.30*
Mean detections / step	4.47	0.60	5.00*
Std. of detections / step	0.73	0.60	0.00*
Steps with zero detections	0	286	0
Both rows found (%)	100.0	0.0	100.0*
Mean Arduino RTT (ms)	9.92	9.91	9.87
Mean vision time (ms)	3.85	3.02	3.42
Mean loop interval (ms)	43.6	48.7	79.6
ACK success (%)	100.0	100.0	100.0
Distinct tracks	10	10	120*

*Reflects a detection lock-on artifact (Section 5). Loop interval excludes the first two steps in every condition. Both-rows fraction is taken over control-log rows.

3.6 Synthetic Weather

Fog and rain are image-space transformations applied to each frame before detection; the detector is otherwise identical across conditions. With intensity $s = 0.6$ for both effects:

- **Fog:** the frame is blended toward a uniform grey (level 205) with weight $0.65s = 0.39$, saturation is multiplied by $1 - 0.5s = 0.7$, and a 3×3 Gaussian blur is applied.
- **Rain:** pixel values are mapped to $0.85\,x - 9$ (scale $1 - 0.25s$, offset $-15s$); 180 short diagonal streaks (length 6–15 px) are drawn and blended with weight $0.35s = 0.21$; and zero-mean Gaussian noise with standard deviation $12s = 7.2$ grey levels is added. The streak positions are drawn once at the start of a run and are static across frames.

The streak positions and noise are generated without a fixed random seed. An intensity of 1.0 for fog caused loss of both rows and a stop in preliminary trials, so 0.6 was used for both effects. Equal intensity does not imply equal perceptual severity. A physically based fog model [8] or rain model [9] would be a more faithful alternative to these filters.

3.7 Experimental Protocol

Each condition was run for 20 s of simulated time (624 control steps). Termination was based on the simulator clock so that all conditions contained the same number of steps. Each condition was executed more than once during development; however, the statistics reported here use one 20 s log per condition. Repeated RAIN executions showed the same qualitative behavior (a constant detection pattern), but those runs were not retained for analysis; consequently, no across-run statistics are reported. The NORMAL log file also contains an earlier aborted run of 68 steps, which was excluded. Detection counts and timing quantities are computed from the per-step log (624 steps); pixel-error statistics, row-found fractions, and track statistics are computed from the control log, which is written every fourth step (156 rows).

4 Results

Table 2 summarizes the results across conditions. The RAIN entries marked with an asterisk are not valid measures of tracking performance, for the reasons given in Section 5.

Fig. 3 overlays the absolute pixel error. Under NORMAL, the error rises gradually and drops abruptly four times (near 4.1, 10.5, 15.6, and 20.0 s), each time the right-row reference changes by 54–169 px, presumably because a different tree becomes the nearest candidate. The controller response is therefore not separable from reference handovers in this signal. FOG has larger and more irregular error. RAIN is nearly flat, which on its own would be read as the best result.

Fig. 4 shows the mean absolute and RMS error. Taken alone, this metric ranks RAIN above NORMAL.

Fig. 5 shows detections per step over all 624 steps. Under NORMAL the count varies between 3 and 8 as trees enter and leave the field of view. Under FOG it lies between 0 and 2, with no detection in 286 of 624 steps (45.8%). Under RAIN it is exactly five in every step.

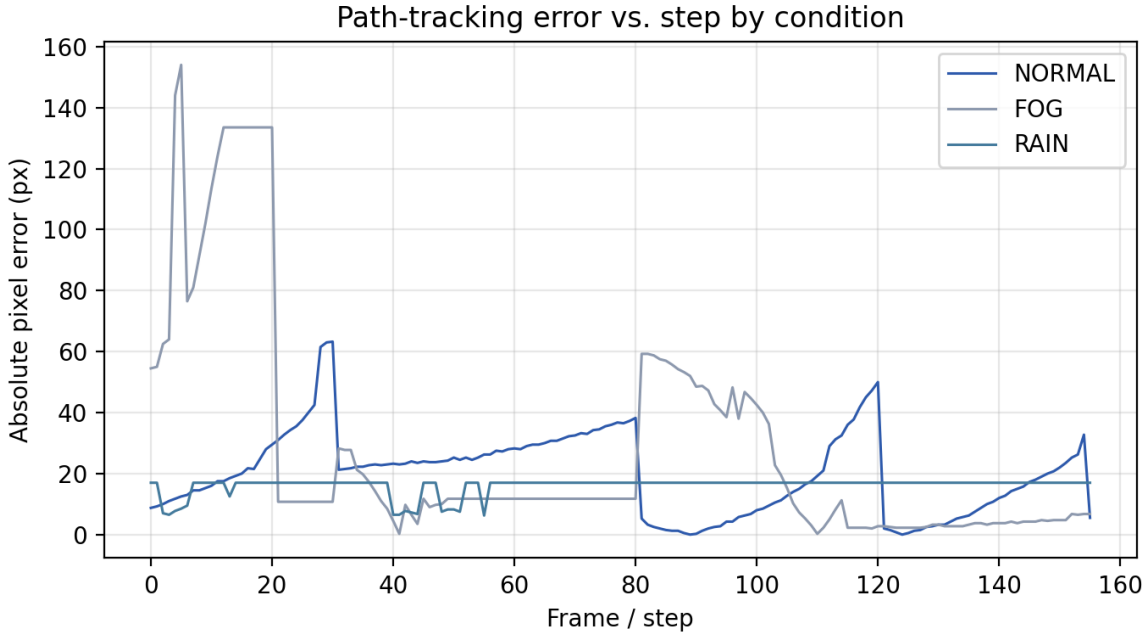


Figure 3: Absolute pixel error versus control step, by condition.

Fig. 6 shows the detector overlay captured from the simulation. Under NORMAL, several trees are boxed and both row references are reported. Under FOG, no tree is detected and the controller uses the stored path centre. Under RAIN, five boxes and both row references are displayed, which appears correct without the diagnostics in Section 5.

5 Discussion: The Rain Lock-On Artifact

During the RAIN run, the robot drove almost straight with little steering correction. Three diagnostics support the conclusion that the detector was not following the scene.

First, the left-row reference is constant at 202 px in all 156 control rows (Fig. 7, left), and the detection count is constant at five in all 624 steps (Fig. 5). Under NORMAL the detection count varies between 3 and 8 over the 624 steps. The NORMAL left reference also stays at 202 px for the first 6.9 s (54 of 156 rows) before drifting, so a constant left reference is not conclusive by itself; it is the combination with a detection count that never changes over the whole run that distinguishes RAIN.

Second, 79% of RAIN detections have a width of exactly 135 px (Fig. 7, right), which is the split-piece width. These boxes are pieces of one oversized region and not individual trees. The split also occurs under NORMAL, where 35% of detections have this width, so a 135 px box alone is not a failure sign; the RAIN signature is that such boxes make up the large majority and all widths lie between 92 and 141 px, whereas 14.5% of NORMAL detections are narrower than 60 px and widths reach 180 px.

Third, the tracker created 120 distinct tracks over the 624 steps under RAIN, with a median lifespan of 3 detection frames, compared with 10 tracks under both NORMAL and FOG.

The most likely cause is an interaction between the rain brightness mapping and the mask's dark-pixel rules. Rain maps a value x to $0.85\,x - 9$. The dark range accepts $V \leq 125$, so after the mapping it accepts original values up to $(125 + 9)/0.85 \approx 158$, and the first dark-trunk rule ($V < 115$) accepts original values up to about 146. Consequently, a larger portion of the scene that was outside the mask in NORMAL can fall inside it. This estimate ignores the streaks and noise and is derived from the thresholds, not measured on the mask images. Under this explanation, trees and background form one wide connected region; the oversized-region split then divides it into 135 px pieces without applying the aspect-ratio test, and the resulting boxes do not move with the scene. Because the path centre is computed from these boxes, it also remains nearly constant, as does the resulting error. Confirming this requires showing the

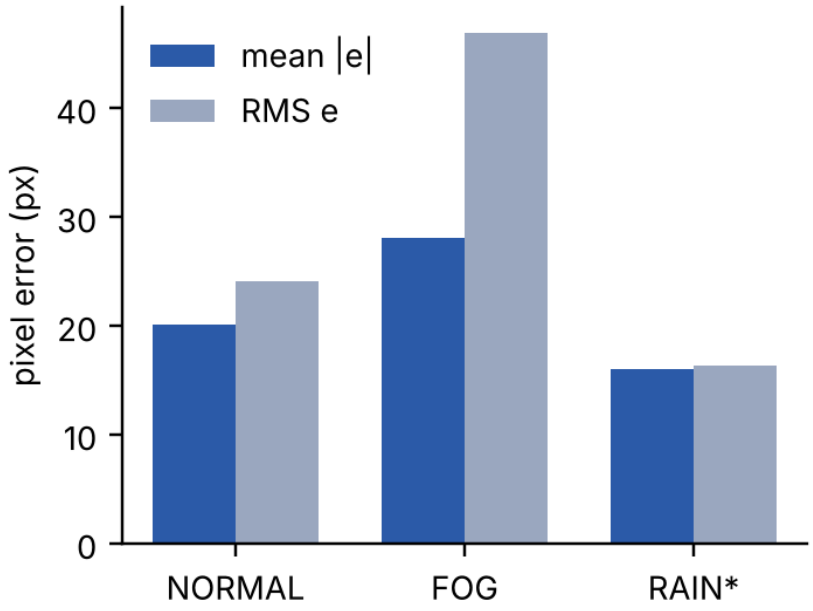


Figure 4: Mean absolute and RMS pixel error by condition. *RAIN values are an artifact (Section 5).

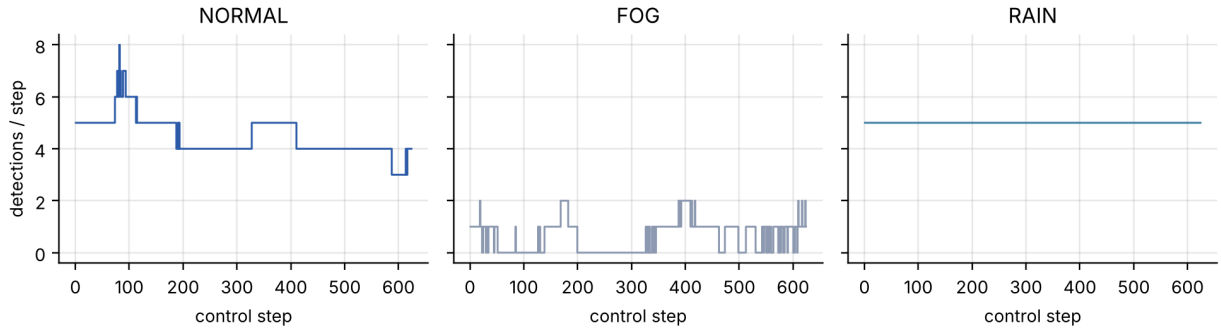


Figure 5: Detections per control step, by condition (all 624 steps).

segmentation masks for the three conditions and re-running RAIN with a brightness-neutral rain model; neither was done here.

The broader point is that, for threshold-based pipelines, a single tracking-error value does not by itself establish weather robustness; in this case, it reversed the apparent ranking of NORMAL and RAIN. We therefore recommend reporting, at minimum, the per-step detection count and its variance, the detection-width distribution, and the track lifespan, and treating an unusually low-variance error signal as a reason to inspect the detector.

6 Latency Analysis

Fig. 8 summarises the timing. The serial round-trip is 9.87–9.92 ms across all conditions, and 100% of commands received a matching acknowledgement. This invariance is expected by construction: the command and acknowledgement contain the same fields in every condition, and serializing roughly 40–50 characters in each direction at 115200 baud accounts for approximately 7–8 ms of the 9.9 ms round-trip. The round-trip therefore serves as a check that the communication path was not a confounding factor; it does not provide evidence about perception. Vision time is 3.0–3.9 ms.

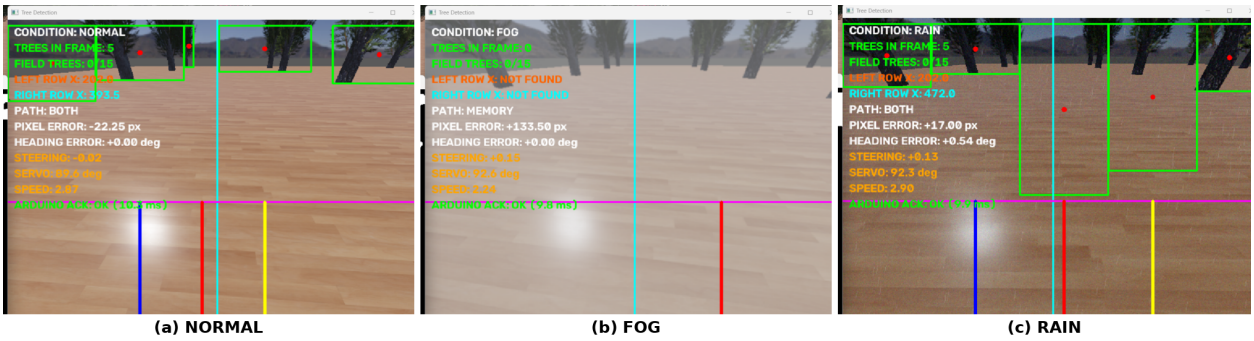


Figure 6: Detector overlay for (a) NORMAL, (b) FOG, and (c) RAIN.

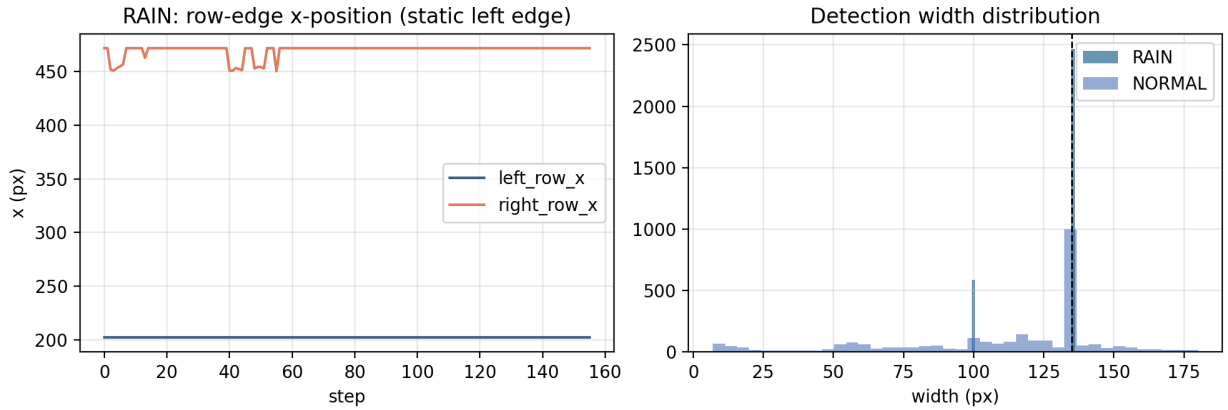


Figure 7: RAIN diagnostics: (left) left row reference position; (right) detection-width distribution with the peak at the 135 px split width.

The loop interval is 43.6 ms (NORMAL), 48.7 ms (FOG), and 79.6 ms (RAIN). In RAIN, the first logged interval after start-up was 3.9 s, corresponding to a single initialization stall that is excluded from the means of all conditions; the median RAIN interval is 78.9 ms. The weather filter runs outside the vision timer and was not timed separately, so the extra RAIN interval cannot be attributed to a specific operation from these logs. Generating and adding the noise array and drawing the streaks for every frame are plausible contributors, but their individual costs were not measured. All intervals exceed the 32 ms simulation step, meaning the simulation runs slower than real time. This does not affect the simulated-time results because the simulator clock advances by one step per iteration, but it would matter for real-time deployment.

7 Limitations and Future Work

Synthetic weather. The weather effects are image-space filters rather than physical atmospheric or droplet models. The rain streak pattern is static within a run, the noise and streak positions are unseeded, and equal intensity does not imply equal severity. A severity sweep and a brightness-neutral rain model would be needed to separate the artifact from rain-like degradation.

One detector. Only a fixed-threshold HSV detector was studied. Whether learned or adaptive-threshold detectors exhibit the same effect is untested; the conclusions are therefore limited to this class of fixed-threshold pipeline under global brightness perturbations.

Hardware coupling. The Arduino acknowledgement does not enter the simulated control loop. The study therefore measures communication latency rather than the effect of actuator delay on tracking.

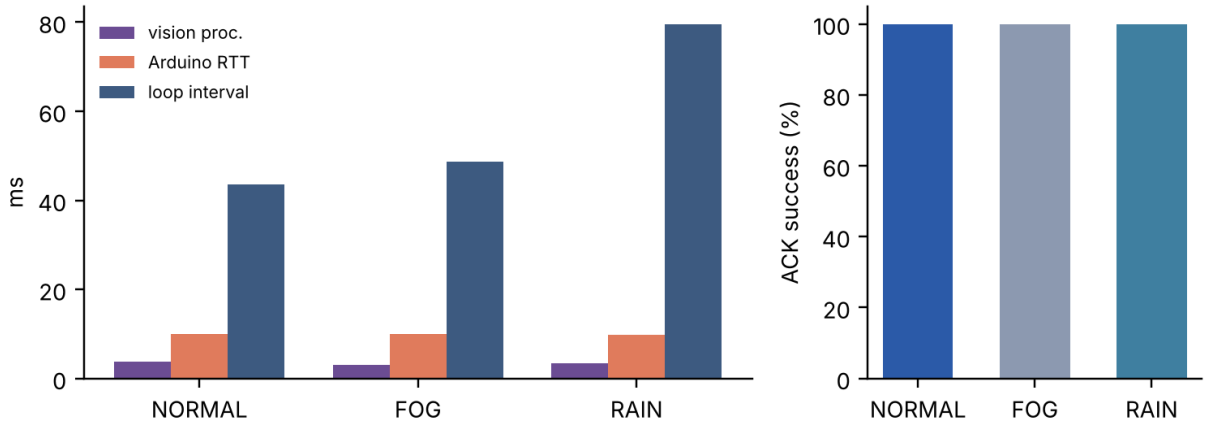


Figure 8: (left) Mean vision time, Arduino round-trip, and loop interval by condition (first two steps excluded); (right) acknowledgement success.

8 Conclusion

A fixed-threshold row detector was evaluated in simulation under clear, foggy, and rainy conditions. Fog reduced detections and increased tracking error. Rain produced the lowest tracking error while the detector output remained effectively constant; detection count, width distribution, and track lifespan identified this behavior as detector lock-on associated with a global brightness reduction interacting with fixed dark-pixel thresholds. For fixed-threshold perception under global brightness perturbations, tracking error should therefore be accompanied by detection-stability diagnostics and, where possible, ground-truth error. The conclusions are based on one log per condition and one detector and should be validated with multi-run statistics and segmentation-level evidence.

References

[1] Björn Åstrand and Albert-Jan Baerveldt. A vision based row-following system for agricultural field machinery. *Mechatronics*, 15(2):251–269, 2005.

[2] Yuhao Bai, Baohua Zhang, Naimin Xu, Jun Zhou, Jiayou Shi, and Zhihua Diao. Vision-based navigation and guidance for agricultural autonomous vehicles and robots: A review. *Computers and Electronics in Agriculture*, 205:107584, 2023.

[3] Alireza Ahmadi, Lorenzo Nardi, Nived Chebrolu, and Cyrill Stachniss. Visual servoing-based navigation for monitoring row-crop fields. In *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, pages 4920–4926, 2020.

[4] Yuxiao Zhang, Alexander Carballo, Hanting Yang, and Kazuya Takeda. Perception and sensing for autonomous vehicles under adverse weather conditions: A survey. *ISPRS Journal of Photogrammetry and Remote Sensing*, 196:146–177, 2023.

[5] Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. In *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.

[6] Claudio Michaelis et al. Benchmarking robustness in object detection: Autonomous driving when winter is coming. *arXiv preprint arXiv:1907.07484*, 2019.

[7] Olivier Michel. Cyberbotics ltd. webots™: Professional mobile robot simulation. *International Journal of Advanced Robotic Systems*, 1(1):39–42, 2004.

[8] Christos Sakaridis, Dengxin Dai, and Luc Van Gool. Semantic foggy scene understanding with synthetic data. *International Journal of Computer Vision*, 126(9):973–992, 2018.

[9] Kshitiz Garg and Shree K. Nayar. Vision and rain. *International Journal of Computer Vision*, 75(1):3–27, 2007.